GUIA PASSO A PASSO

# Pare de ensinar a mesma coisa para a IA.

autoharness, da Tigerless Labs: o Claude Code que guarda como você trabalha e o porquê de cada decisão.

O QUE É

# Uma skill que se escreve sozinha.

**Skill**
uma instrução salva que a IA carrega quando o assunto aparece, como o seu jeito de montar um relatório

**Aprende**
transforma as suas sessões reais em skills, sem você escrever nada

**Organiza**
junta skills do mesmo cenário, atualiza com o uso e descarta as que não são usadas

**Só as dele**
mexe apenas nas skills que ele mesmo criou; as suas ficam intactas

REQUISITOS

# Antes de instalar.

### 1. Confira a versão do Python

```
python3 --version
```

Precisa ser 3.11 ou mais novo, e com o nome python3 no PATH.

### 2. Se aparecer 3.9

```
which python3 # /usr/bin/python3 é o do Xcode
```

O Python 3.9 do Xcode desliga todos os hooks da sessão, e ele avisa no stderr. Instale um 3.11+ (por exemplo, pelo Homebrew) e deixe-o antes no PATH.

### 3. Sistema

```
Mac ou Linux
```

O selo do README lista Linux e macOS. Windows não aparece como suportado.

PASSO 1 · INSTALAR

# Três comandos no Claude Code.

**Digite na caixa de texto do Claude Code**

```
/plugin marketplace add tigerless-labs/autoharness
/plugin install autoharness@autoharness
```

O primeiro registra o catálogo; o segundo instala o plugin.

**Depois recarregue**

```
/reload-plugins
```

Ou feche e abra o Claude Code de novo.

Dica: faça isso numa pasta de teste, não no projeto do trabalho.

DEPOIS DE INSTALAR

# Você não precisa fazer nada.

### Zero config

ele já observa as suas sessões e grava as skills aprendidas em segundo plano

### Quando aprende

depois de 50 chamadas de ferramenta, a sessão termina com uma reflexão em segundo plano

### Onde ficam

em .claude/skills/ no projeto, ou em ~/.claude/skills/ para o que vale em qualquer projeto

### Começo da sessão

toda sessão nova abre com um índice das skills dele, agrupadas por categoria

Conversa sem trabalho não dispara nada: só conta chamada de ferramenta.

O COMANDO /LEARN

# Guardar a lição agora.

Digite `/learn` depois de resolver algo e ele destila a sessão em que você está, sem esperar o ciclo automático.

**Exemplo ilustrativo**

```
você: o relatório semanal sai errado se eu incluir
o fim de semana, porque o time fecha na sexta.
(Claude ajusta e o relatório fica certo)
/learn
```

A lição passa pela mesma cadeia de proposta e validação do aprendizado em segundo plano. Na sessão seguinte, o porquê está no índice.

REVISAR, EDITAR, APAGAR

# Tudo são arquivos comuns.

### Ver o que ele criou

```
ls -la .claude/skills/
cat .claude/skills/<nome>/SKILL.md
```

Use ls -la: os arquivos interessantes são ocultos. A skill é um SKILL.md no formato nativo.

### Ver por que nasceu

```
cat .claude/skills/<nome>/.ledger.jsonl
```

Uma linha por evento (create, patch...), com o motivo e o trecho da sessão que a justificou.

### Descartar sem perder

```
mv .claude/skills/<nome> .claude/skills/.archive/
```

É o caminho que o próprio projeto usa para aposentar uma skill. Para voltar, mova a pasta de volta. Editar o SKILL.md à mão também é possível.

PASSO 2 · CONFIGURAR (OPCIONAL)

# O ritmo é ajustável.

### Aprender mais rápido (padrão 50)

```
export AUTOHARNESS_REFLECT_EVERY_N=10
```

Menos chamadas por ciclo: aprende mais cedo e abre mais sessões filhas.

### Só neste projeto, em .claude/settings.json

```
{ "env": { "AUTOHARNESS_DISABLE_GLOBAL": "1" } }
```

Nada de skill global. Mudança vale a partir da próxima sessão.

### Sem índice no começo

```
export AUTOHARNESS_INDEX_SUSPENDED=1
```

Para quem não quer gastar contexto com o índice. O resto continua rodando.

Outras: AUTOHARNESS_NOTIFY=desktop avisa a cada skill criada.

EXEMPLO 1 · RELATÓRIO SEMANAL

# O seu processo, lembrado.

EXEMPLO ILUSTRATIVO

### Você trabalha normal e explica

> “Puxe as métricas de segunda a sexta, compare com a semana anterior e destaque só o que mudou mais de 10%.”

Ao longo da sessão, você corrige e diz por que: o time fecha na sexta.

### O que ele pode guardar

```
skill: relatorio-semanal-metricas
regra: janela seg a sex; destaca variação acima de 10%;
o motivo
```

Na próxima sessão, a regra e o porquê já estão no índice.

EXEMPLOS 2 E 3 · SQL E PR

# Padrões do time, sem repetir.

EXEMPLOS ILUSTRATIVOS

### Padrão de consulta SQL

```
“Sempre filtre pedidos cancelados e use a tabela de
fatos, não a de log.”
```

Pode virar uma skill de consulta, com o motivo da escolha da tabela.

### Formato de PR do time

```
“PR com contexto, métrica de sucesso e risco, nessa
ordem.”
```

Pode virar uma skill de formato de PR. Se o time mudar a ordem, a skill é atualizada, não duplicada.

EXEMPLO 4 · CHECKLIST DE REVISÃO

# O que você sempre confere.

EXEMPLO ILUSTRATIVO

**Você revisa um documento e repete**

> “Antes de aprovar: dono definido, prazo, métrica e plano de volta.”

Depois de algumas sessões, ele pode juntar isso numa skill só de checklist.

**Se a regra mudar**

> “Agora também precisa de aprovação do jurídico.”

O reflector deve reescrever a skill antiga na mesma rodada, em vez de deixar duas versões em conflito.

DESINSTALAR E ATUALIZAR

# Sair não apaga o que ele criou.

### Desinstalar, no terminal

```
claude plugin uninstall autoharness@autoharness
claude plugin marketplace remove autoharness
```

### Limpar o que sobrou

```
rm -r ~/.claude/autoharness/ .claude/autoharness/
rm -r as skills marcadas self-authored
```

As skills e o estado ficam no disco mesmo depois de desinstalar. Cada skill dele tem o marcador self-authored no ledger.

### Atualizar, no terminal

```
claude plugin marketplace update autoharness
claude plugin update autoharness@autoharness
```

Depois reinicie o Claude Code.

CUIDADOS

# Teste num projeto de teste primeiro.

Repositório novo (junho de 2026), mesmo com muitas estrelas. Ele roda hooks em toda sessão e grava arquivos em segundo plano: revise o que ele cria. O aprendizado abre sessões filhas, então pode haver uso extra; o README não dá o número. Os 42% para 78% são do próprio projeto.

Quem quiser usar, o passo a passo está aqui. autoharness, licença MIT. O link está no primeiro comentário.